

Unit 2: Big Data Infrastructures (Compute/Storage) — **Question Bank**

4 Mark Long-Answer Questions

Topics covered: Hadoop Ecosystem, Apache Hive, Apache HBase, Apache Cassandra, Apache Iceberg, Apache SeaTunnel, Apache Druid, Apache Hudi, Apache Airflow

50 Questions

(Q101–Q130)

Q101. [4 marks]

Explain the roles of HDFS and YARN in the Hadoop ecosystem.

Ans:

- HDFS (Hadoop Distributed File System) is Hadoop's storage layer — it stores large files as blocks that are distributed and replicated across DataNodes for fault tolerance. (1)
- The NameNode manages metadata such as the namespace and block locations, while DataNodes physically store the block data. (1)
- YARN is the resource management and compute layer — it allocates cluster resources such as CPU and memory to running applications. (1)
- YARN's ResourceManager (cluster-wide) and NodeManagers (per-node) work together to schedule and monitor containers, decoupling resource management from the specific processing engine, whether that is MapReduce, Spark, or Hive. (1)

Q102. [4 marks]

Describe the MapReduce programming model with its two main phases.

Ans:

- MapReduce processes large datasets in parallel across a cluster using two main phases: Map and Reduce. (1)
- In the Map phase, input data is split across mapper tasks, each of which transforms its portion of input into intermediate key-value pairs. (1)
- In the Shuffle/Sort phase, intermediate pairs are grouped and sorted by key, and then distributed to the appropriate reducer tasks. (1)
- In the Reduce phase, each reducer aggregates or summarizes the values associated with its keys, producing the final output, which is typically written back to HDFS. (1)

Q103. [4 marks]

What is Apache Hive? Explain why it is useful for big data analysts.

Ans:

- Hive is a data warehousing tool built on top of Hadoop that provides a SQL-like query language called HiveQL for querying large datasets. (1)
- It abstracts away the complexity of writing low-level distributed processing code, such as raw MapReduce or Spark jobs. (1)
- It supports a schema-on-read model, letting users impose structure on data already stored in HDFS or other storage systems. (1)
- It is especially useful because analysts who already know SQL can query massive datasets without learning distributed programming, and Hive integrates readily with common BI and reporting tools. (1)

Q104. [4 marks]

Differentiate between Hive's managed (internal) and external tables.

Ans:

- For managed tables, Hive owns both the data and the metadata, and the data resides inside Hive's default warehouse directory. (1)
- Dropping a managed table deletes both its metadata definition and its underlying data files. (1)
- For external tables, Hive manages only the metadata, while the actual data resides at a location the user specifies outside the default warehouse. (1)
- Dropping an external table removes only the metadata, leaving the underlying data intact, which is useful when the same data needs to be shared across multiple tools. (1)

Q105. [4 marks]

List and briefly explain any four features of Apache HBase.

Ans:

(1 mark each for any four, described in sentence form, e.g.)

- Column-oriented storage: HBase organizes data into column families rather than fixed rows, which suits sparse and wide datasets well.
- Horizontal scalability: additional RegionServers can be added to handle growing data volumes and request load.
- Strong consistency: HBase guarantees strong, atomic consistency for read and write operations on a single row.
- Real-time random access: unlike batch-oriented Hive, HBase supports fast random reads and writes on individual records.
- Reliance on HDFS: HBase leverages HDFS underneath for durable, distributed, and replicated storage of its data files.

Q106. [4 marks]

Explain the concept of "row key" in HBase and its importance in design.

Ans:

- The row key uniquely identifies a row in an HBase table, and rows are physically stored in lexicographically sorted order by their row key. (1)
- All read and write operations, such as Get, Scan, and Put, are indexed and optimized around the row key. (1)
- A poorly designed row key, such as a monotonically increasing sequence, can cause "hotspotting," where writes concentrate on a small number of RegionServers. (1)
- Techniques such as salting, hashing, or reversing keys are commonly used to distribute writes more evenly across regions and improve overall throughput. (1)

Q107. [4 marks]

What is Apache Cassandra? Mention its key characteristics.

Ans:

- Cassandra is a distributed, wide-column NoSQL database designed for high availability and scalability, with no single point of failure. (1)
- It uses a masterless, peer-to-peer architecture in which every node in the cluster performs an identical role. (1)
- It offers tunable consistency, allowing applications to balance consistency against availability on a per-operation basis. (1)
- It is particularly well optimized for write-heavy workloads and scales linearly simply by adding more nodes to the cluster. (1)

Q108. [4 marks]

Explain the concept of "partition key" and "clustering columns" in Cassandra.

Ans:

- The partition key determines which node or nodes will store a given row, based on hashing the key to a position (token) on the Cassandra ring. (1)
- All rows sharing the same partition key are stored together on the same node(s), which supports efficient retrieval of related data. (1)
- Clustering columns determine the sort order of rows within a given partition, enabling efficient range queries within that partition. (1)
- Together, the partition key and clustering columns form the primary key and heavily influence both data modeling decisions and query performance in Cassandra. (1)

Q109. [4 marks]

What problems does Apache Iceberg solve for traditional Hive tables on data lakes?

Ans:

- Traditional Hive tables lack safe schema evolution, since changing a column's type or position can silently break existing readers of the data. (1)
- They also lack ACID transaction guarantees, which risks partial or inconsistent reads occurring during concurrent write operations. (1)
- Hive's directory-based partitioning requires users to know and reference the physical layout, and changing that layout later is costly. (1)
- Iceberg addresses these issues through a dedicated metadata layer that supports atomic commits, safe schema evolution, and hidden partitioning. (1)

Q110. [4 marks]

Describe the layered architecture (metadata layers) of Apache Iceberg.

Ans:

- The catalog layer maps a table's name to the location of its current metadata file, and atomically updating this mapping is what makes commits safe. (1)
- The metadata file stores the table's schema, its partition specification, and the list of all snapshots taken over time. (1)
- The manifest list, associated with each snapshot, records which manifest files make it up, along with summary-level statistics. (1)
- The manifest files themselves list the individual data files, along with partition values and column-level statistics used to prune irrelevant files during query planning. (1)

Q111. [4 marks]

What is Apache SeaTunnel, and what problem does it solve?

Ans:

- SeaTunnel is a distributed, high-performance data integration tool designed to synchronize data between many different sources and sinks. (1)
- It solves the problem of having to build and maintain numerous custom, point-to-point ETL scripts between heterogeneous systems. (1)
- It offers a unified, pluggable connector framework, structured around a Source-Transform-Sink model, that supports hundreds of different data source and sink types. (1)
- It is specifically designed to scale to very large data volumes by leveraging distributed execution engines. (1)

Q112. [4 marks]

Explain the Source-Transform-Sink pipeline model used by data integration tools like SeaTunnel.

Ans:

- The Source stage connects to and reads data from an originating system, such as a database, file, or message queue. (1)
- The Transform stage applies operations such as filtering, mapping, or cleansing to the extracted data before it moves further along the pipeline. (1)
- The Sink stage writes the transformed data out to its final destination system. (1)
- This modular design allows connectors to be reused across many different pipelines and supports flexible many-to-many synchronization between diverse systems. (1)

Q113. [4 marks]

What is Apache Druid, and what types of workloads is it designed for?

Ans:

- Druid is a high-performance, column-oriented, distributed analytics database. (1)
- It is explicitly designed to support both real-time streaming ingestion and historical batch data within the same system. (1)
- It particularly excels at fast, interactive OLAP-style aggregation queries, which are commonly used to power monitoring and analytics dashboards. (1)
- It effectively combines ideas from traditional data warehouses, time-series databases, and search systems into a single unified engine. (1)

Q114. [4 marks]

List the core node types in a Druid cluster and their basic roles.

Ans:

(1 mark each for any four, in sentence form, e.g.)

- The Coordinator manages the distribution and balancing of segments across Historical nodes.
- The Overlord manages the assignment and lifecycle of ingestion tasks, such as data-loading jobs.
- The Broker routes incoming queries to the appropriate nodes and merges their results before returning a final response.
- Historical nodes store finalized, immutable segments and serve queries directly against them.
- MiddleManager nodes execute ingestion tasks, including handling real-time streaming ingestion.

Q115. [4 marks]

What is Apache Hudi, and what core capability does it add to data lakes?

Ans:

- Hudi, short for Hadoop Upserts Deletes and Incrementals, is a storage framework designed for data lakes. (1)
- It enables record-level upserts and deletes on data stored in formats like Parquet, unlike plain, immutable file storage. (1)
- It supports incremental data processing, allowing downstream consumers to pull only the records that have changed since a given point in time. (1)
- It integrates with popular query engines such as Spark, Hive, and Presto, enabling near real-time analytics directly on data lake storage. (1)

Q116. [4 marks]

Differentiate between Copy-on-Write (COW) and Merge-on-Read (MOR) table types in Hudi.

Ans:

- Copy-on-Write tables merge updates into new, complete versions of the base columnar files as soon as a write occurs. (1)
- This design offers faster reads, since no merging is needed at query time, but comes at the cost of more expensive and slower writes. (1)
- Merge-on-Read tables instead append updates as separate delta log files, which are merged with the base files only at read time or during a background compaction process. (1)
- This design offers faster writes, at the cost of potentially slower reads until compaction has run to consolidate the logs. (1)

Q117. [4 marks]

What is Apache Airflow, and why is workflow orchestration important in big data pipelines?

Ans:

- Airflow is a platform for programmatically authoring, scheduling, and monitoring workflows, which are represented internally as DAGs. (1)
- Big data pipelines typically involve many interdependent steps, such as extraction, transformation, loading, and validation. (1)
- Orchestration ensures these tasks run in the correct order, automatically handles retries and failures, and provides visibility into pipeline health. (1)
- Airflow's DAG-based model, together with its operators, sensors, and hooks, makes complex dependencies between pipeline steps explicit and much easier to manage. (1)

Q118. [4 marks]

Explain the concept of a DAG (Directed Acyclic Graph) in the context of Airflow.

Ans:

- A DAG is a graph structure consisting of directed edges that contains no cycles anywhere within it. (1)
- In Airflow, each node in the DAG represents a task, and each directed edge represents a dependency that dictates task execution order. (1)
- Being "acyclic" ensures that no task can ever depend on itself, whether directly or indirectly, which prevents infinite loops in the workflow. (1)
- DAGs are defined as Python scripts, which allows workflows to be built dynamically and programmatically, along with associated scheduling metadata. (1)

Q119. [4 marks]

Explain the difference between batch processing and real-time (streaming) processing, referencing at least one Big Data tool for each.

Ans:

- Batch processing handles large volumes of data that have been collected over a period of time and are then processed together as a single unit. (1)
- For example, Hive queries executed via MapReduce typically run as scheduled batch jobs over data that has already been stored. (1)
- Real-time or streaming processing instead handles data continuously as it arrives, aiming to keep processing latency low. (1)
- For example, Apache Druid supports real-time ingestion, making streaming data queryable almost as soon as it arrives, alongside previously stored historical data. (1)

Q120. [4 marks]

What is meant by "schema evolution," and why is it important for data lake table formats like Iceberg?

Ans:

- Schema evolution refers to the ability to change a table's structure over time, such as adding, dropping, renaming, or reordering columns, or changing column types. (1)
- Data lakes tend to accumulate data over many years, and both business requirements and upstream data sources change frequently during that time. (1)
- Without safe schema evolution, such changes risk breaking existing readers of the data or requiring an expensive rewrite of the entire table. (1)
- Iceberg tracks schema changes explicitly in its metadata layer, which allows schemas to evolve safely without needing to rewrite any existing data files. (1)

Q121. [4 marks]

Explain why NoSQL databases like HBase and Cassandra were developed, in contrast to traditional RDBMS.

Ans:

- Traditional relational databases often struggle to scale horizontally across many commodity servers when handling truly massive datasets. (1)
- RDBMS systems also enforce rigid schemas and strong ACID guarantees, which can limit throughput and availability at very large scale. (1)
- NoSQL databases such as HBase and Cassandra were developed to prioritize horizontal scalability, more flexible schemas, and high availability instead. (1)
- They were specifically designed to handle the volume, velocity, and variety characteristic of big data workloads, which traditional RDBMS were not originally built to address. (1)

Q122. [4 marks]

Describe the basic write path in HBase (from client write to persistent storage).

Ans:

- The client sends a Put request, which is routed to the appropriate RegionServer based on the target row key. (1)
- The write is first appended to the Write-Ahead Log (WAL), which ensures durability in case of a subsequent failure. (1)
- The write is then applied to the in-memory MemStore associated with the relevant column family. (1)
- Once the MemStore reaches a configured size threshold, its contents are flushed to disk as a new, immutable HFile. (1)

Q123. [4 marks]

Explain the concept of "eventual consistency" and how Cassandra applies tunable consistency.

Ans:

- Eventual consistency means that, given enough time with no further updates, all replicas of a data item will eventually converge to hold the same value. (1)
- Cassandra allows clients to select a Consistency Level, such as ONE, QUORUM, or ALL, individually for each read or write operation. (1)
- Choosing a lower consistency level like ONE favors availability and lower latency, while a higher level like QUORUM or ALL favors stronger consistency. (1)
- This tunability lets applications navigate the trade-offs described by the CAP theorem in a way that fits their specific requirements. (1)

Q124. [4 marks]

What are "snapshots" in Apache Iceberg, and what benefits do they provide?

Ans:

- A snapshot represents the complete state of an Iceberg table as it existed at one particular point in time. (1)
- Every write operation, whether an insert, update, or delete, produces a new snapshot rather than modifying the existing data in place. (1)
- Snapshots enable "time travel" queries, allowing users to query the table exactly as it appeared at some previous snapshot or timestamp. (1)
- They also support safely rolling back the table to a previous, known-good state if an erroneous write is later discovered. (1)

Q125. [4 marks]

Explain the importance of connectors in data integration tools such as SeaTunnel.

Ans:

- Connectors are pluggable modules that allow a tool to read from a source system or write to a sink system. (1)
- They abstract away system-specific details, such as APIs, protocols, and data formats, behind a common, consistent interface. (1)
- A rich connector ecosystem allows a tool like SeaTunnel to integrate with hundreds of databases, files, and streaming systems without writing custom code for each individual case. (1)
- This modularity increases reusability across pipelines, reduces development effort, and simplifies ongoing maintenance of integration workflows. (1)

Q126. [4 marks]

Explain "roll-up" in Apache Druid and its benefit.

Ans:

- Roll-up is a summarization technique applied at ingestion time, where rows sharing the same dimension values and time granularity are aggregated together into one row. (1)
- Instead of storing every individual raw row, aggregate metrics such as sums or counts are computed and stored in its place. (1)
- This significantly reduces the total number of stored rows, which in turn saves considerable storage space. (1)
- It also improves query performance, since fewer rows need to be scanned to answer typical aggregation-style queries. (1)

Q127. [4 marks]

What are "incremental pipelines," and how does Apache Hudi support them?

Ans:

- Incremental pipelines process only the data that has changed since the previous run, rather than reprocessing the entire dataset every time. (1)
- This approach substantially reduces compute cost and processing time when working with very large datasets. (1)
- Hudi maintains a detailed timeline of commits, which allows downstream consumers to query only the records that changed after a given commit or timestamp. (1)
- This capability enables near real-time ETL and analytics pipelines to be built directly on top of the data lake. (1)

Q128. [4 marks]

Explain the role of "Operators," "Hooks," and "Sensors" in Apache Airflow.

Ans:

- Operators define a single, atomic unit of work performed by a task; for example, a BashOperator runs a shell command, while a PythonOperator runs Python code. (2)
- Hooks provide reusable interfaces for interacting with external systems, such as databases, APIs, or cloud services, and are typically used from within operators. (1)
- Sensors are a special category of operator that wait or poll for a specific condition, such as a file's arrival or a table update, before allowing downstream tasks to proceed. (1)

Q129. [4 marks]

Compare Hive and HBase in terms of their intended use cases.

Ans:

- Hive is designed for running batch analytical queries over very large datasets using SQL-like syntax, prioritizing overall throughput rather than low latency. (1)
- HBase is instead designed for real-time random read and write access to individual records at scale, prioritizing low latency. (1)
- Hive is typically well suited to reporting, ad-hoc analytics, and ETL workloads, while HBase suits operational workloads that need fast lookups, such as a serving layer for an application. (1)
- Although both systems commonly run on top of HDFS, they are built to serve fundamentally different access patterns. (1)

Q130. [4 marks]

Give a short overview of how each of the following fits in a modern big data pipeline: Airflow, Hive, Druid.

Ans:

- Airflow orchestrates the overall pipeline, scheduling and sequencing the various ETL jobs that make up the workflow. (2)
- Hive is used for batch transformation and analytical SQL querying of data that has already landed in the data lake or warehouse. (1)
- Druid serves as a fast analytics and serving layer, powering real-time dashboards on top of the processed and ingested data. (1)

(Q131–Q143)

Q131. [4 marks]

Explain how HDFS achieves fault tolerance, referencing replication and the role of the NameNode/DataNodes.

Ans:

- HDFS replicates each block, by default three times, across multiple DataNodes, often spreading these replicas across different racks. (1)
- If a DataNode fails, the NameNode detects the missing heartbeats and marks that DataNode as dead. (1)
- The NameNode then instructs the remaining healthy DataNodes to re-replicate any blocks that fall below the desired replication factor. (1)
- Checksums are also used to detect corrupted blocks, and rack-aware placement helps balance the trade-off between fault tolerance and network overhead. (1)

Q132. [Numerical; 4 marks]

Discuss the trade-offs between using MapReduce and Apache Tez/Spark as Hive's execution engine, and briefly explain, with an illustrative example, why avoiding intermediate disk writes matters for a 5-stage query.

Ans:

- MapReduce writes intermediate results to disk between each stage of processing, which causes higher latency for multi-stage queries. (1)
- Tez instead models a query as a single DAG of tasks, avoiding unnecessary disk writes between stages and generally improving performance over classic MapReduce. (1)
- Spark, used as an execution engine, leverages in-memory computation, often providing further speedups, particularly for iterative or multi-stage queries. (1)
- As an illustration, a 5-stage MapReduce query might perform 4 rounds of disk-based intermediate writes and reads between stages, whereas Tez/Spark can often keep intermediate data in memory across all 5 stages, substantially cutting down total I/O and end-to-end query latency, at the cost of requiring more cluster memory. (1)

Q133. [4 marks]

Explain HBase's read path, including how it uses MemStore, BlockCache, and HFiles.

Ans:

- On receiving a Get request, HBase first checks the BlockCache, which holds an in-memory cache of frequently accessed data blocks. (1)
- It also checks the MemStore for any very recent, still-unflushed writes that match the requested row. (1)
- If the data is not found in memory, HBase reads from the relevant HFiles on disk, using Bloom filters to skip files that are guaranteed not to contain the requested key. (1)
- The results retrieved from the MemStore and the HFiles are merged, using timestamps to determine and return the most recent version of the requested cell(s). (1)

Q134. [4 marks]

Discuss Cassandra's replication strategy and the role of the "replication factor."

Ans:

- The replication factor (RF) defines how many copies of each row are stored across the cluster. (1)
- SimpleStrategy places replicas without any awareness of data center topology, which suits single-data-center deployments. (1)
- NetworkTopologyStrategy allows specifying a distinct replication factor for each data center in multi-data-center deployments, improving both fault tolerance and data locality. (1)
- Combined with tunable consistency levels, the replication factor is a key lever that determines the overall trade-off between availability, consistency, and fault tolerance. (1)

Q135. [4 marks]

Explain Iceberg's "hidden partitioning" and how it differs from Hive-style partitioning.

Ans:

- In Hive, users must explicitly include partition columns in their query predicates in a way that matches the physical partition layout, such as writing WHERE year=2024. (1)
- If the physical layout is later changed, existing queries can break entirely or become significantly less efficient. (1)
- Iceberg's hidden partitioning instead applies partition transforms, such as bucketing, truncation, or date-based transforms, automatically behind the scenes, without requiring users to know or reference the physical partition columns. (1)
- Iceberg can also evolve its partition specification over time without requiring a full rewrite of the table, which is not possible with Hive-style partitioning. (1)

Q136. [4 marks]

Describe how SeaTunnel achieves high-performance distributed data synchronization.

Ans:

- SeaTunnel can run on distributed execution engines, such as Spark or Flink, or on its own dedicated Zeta engine, parallelizing data movement across cluster nodes. (1)
- Its pluggable connector architecture allows parallel reads and writes across many source and sink partitions simultaneously. (1)
- It supports checkpointing and fault tolerance, allowing large synchronization jobs to resume from a checkpoint rather than restarting entirely after a failure. (1)
- Its configuration-driven pipelines, rather than fully custom code, reduce development overhead while still scaling effectively to very large data volumes. (1)

Q137. [4 marks]

Explain how Druid handles both real-time streaming data and historical batch data within a single system.

Ans:

- Real-time ingestion tasks, run by MiddleManager nodes, continuously ingest streaming data (for example, from Kafka) and make it immediately available for querying. (1)
- As this real-time data ages, it is converted into immutable segments and handed off to Historical nodes for long-term storage. (1)
- Broker nodes route incoming queries to both real-time and Historical nodes as needed and merge the results into a single, unified response. (1)
- This overall architecture allows Druid to provide fast queries spanning both the very newest data and years of accumulated historical data within one system. (1)

Q138. [4 marks]

Explain the trade-offs between Copy-on-Write and Merge-on-Read table types in Hudi, and suggest when each is appropriate.

Ans:

- Copy-on-Write merges updates into new base files immediately at write time, so reads remain fast since they query base files directly, but write operations become more expensive and slower. (1)
- COW is therefore well suited to read-heavy workloads where write latency is less critical, such as periodic batch updates. (1)
- Merge-on-Read appends changes as small log files, which are merged with base data only at read time or during compaction, so writes are fast but reads may be slower until compaction has occurred. (1)
- MOR is therefore better suited to write-heavy, near real-time ingestion scenarios where minimizing write latency matters more than immediate read speed. (1)

Q139. [4 marks]

Explain how Airflow's Scheduler and Executor work together to run a DAG.

Ans:

- The Scheduler continuously parses DAG files, checks their schedules, and determines which task instances are ready to run based on their dependencies. (1)
- Task instances that are ready to run are queued by the Scheduler and then handed off to the Executor. (1)
- The Executor determines how and where these tasks actually run; for example, a LocalExecutor runs tasks as local subprocesses, while a CeleryExecutor distributes tasks to remote worker machines. (1)
- The resulting task state, whether success, failure, or retry, is recorded in the metadata database, which the Scheduler then consults to correctly manage dependent downstream tasks. (1)

Q140. [4 marks]

Compare Apache Druid and Apache Hive for analytical workloads, highlighting when each is preferable.

Ans:

- Hive is optimized for large-scale batch analytical queries that offer high throughput but tend to have relatively high latency, ranging from seconds to minutes. (1)
- Druid is instead optimized for sub-second, interactive OLAP queries, especially for aggregations performed over time-series or event data. (1)
- Hive generally supports more complex, ad-hoc SQL operations such as joins and deep analytics better than Druid, which favors simpler, very fast aggregation-style queries. (1)
- As a result, Druid tends to be preferable for real-time dashboards, while Hive is generally preferable for deeper, exploratory batch analysis and ETL work. (1)

Q141. [4 marks]

Explain the concept of "compaction" in the context of both Cassandra and Apache Hudi.

Ans:

- In Cassandra, compaction merges multiple SSTables together into fewer, larger SSTables, removing obsolete or deleted (tombstoned) data in the process. (2)
- In Hudi, specifically for Merge-on-Read tables, compaction merges the accumulated delta log files with base files, producing updated, query-optimized base files. (1)
- In both systems, compaction is a background process that trades some additional I/O and CPU usage for improved long-term read efficiency and reduced storage bloat. (1)

Q142. [4 marks]

Explain how Iceberg supports concurrent writers safely without locking the entire table.

Ans:

- Iceberg relies on optimistic concurrency control: a writer reads the current table metadata, makes its intended changes, and produces a new metadata file representing a new snapshot. (1)
- To commit these changes, the writer attempts to atomically swap the catalog's pointer from the old metadata file to the new one, but only if the pointer has not changed since it was originally read. (1)
- If another writer has already committed a conflicting change in the meantime, the commit attempt fails, and the writer must retry against the newly updated table state. (1)
- This approach avoids the need to lock the entire table while still guaranteeing that every commit remains atomic and consistent. (1)

Q143. [4 marks]

Design a brief data pipeline using Airflow, Hive, and HBase for a use case where daily batch sales data must be transformed and also made available for fast lookups.

Ans:

- An Airflow DAG orchestrates the overall daily pipeline, triggered on a fixed schedule such as midnight each day. (1)
- One task ingests the raw sales data into HDFS, followed by a Hive task that transforms, cleans, and aggregates the data using HiveQL. (1)
- A subsequent task loads the transformed, per-record data into HBase, enabling fast, low-latency lookups by key, such as an order ID or customer ID. (1)
- Airflow manages the dependencies and retries between these different stages, ensuring the entire pipeline runs reliably from start to finish. (1)

(Q144–Q150)

Q144. [Numerical; 4 marks]

A cluster stores 10 TB of unique data in HDFS. Analyze the trade-offs of using a replication factor of 2 versus a replication factor of 3 in terms of storage cost, fault tolerance, and read performance, including the total physical storage each option requires.

Ans:

- With a replication factor of 2, the cluster would need to provision roughly 20 TB of total physical storage ($10 \text{ TB} \times 2$), while a replication factor of 3 would require roughly 30 TB ($10 \text{ TB} \times 3$) — a 10 TB difference in storage cost. (1)
- The higher replication factor of 3 improves fault tolerance, since two simultaneous node/block failures can be tolerated before any data is lost, compared to only one failure being tolerable at a replication factor of 2. (1)
- A higher replication factor can also improve read performance and throughput, since more nodes are available to serve read requests in parallel for frequently accessed blocks. (1)
- The lower replication factor of 2 reduces storage cost significantly but increases the risk of data loss and can create read bottlenecks for popular data; the right choice ultimately depends on the criticality of the data and the size/resilience of the cluster. (1)

Q145. [4 marks]

Critically evaluate row key design strategies in HBase (sequential vs. salted/ hashed keys) for a time-series write-heavy workload.

Ans:

- Sequential row keys, such as those based directly on a timestamp, cause all new writes to target the same region and RegionServer at any given moment, creating a "hotspot" that limits overall write throughput. (2)
- Salting, which prefixes keys with a hash or random value, distributes writes evenly across regions, but it complicates range scans since logically related data ends up scattered across different regions. (1)
- Reversing or bucketing timestamps can balance write distribution while partially preserving scan locality, although this approach adds extra design complexity. (1)
- Ultimately, the right strategy depends on whether the workload prioritizes evenly distributed write throughput or efficient range scans over time-ordered data.

Q146. [4 marks]

Discuss how Cassandra's eventual consistency model can lead to stale reads, and explain how consistency levels and repair mechanisms mitigate this.

Ans:

- Because writes are replicated asynchronously across replicas, a read directed at a replica that has not yet received the latest write can return stale data. (1)
- Using higher consistency levels, such as QUORUM for both reads and writes, ensures that overlapping sets of replicas are checked, which reduces (though does not fully eliminate) the chance of staleness. (1)
- Read repair automatically reconciles replicas that return divergent data during the course of a normal read operation. (1)
- Anti-entropy repair, such as running the nodetool repair command, periodically synchronizes replicas across the entire cluster to fix inconsistencies that may have arisen from missed writes or failed hinted handoffs. (1)

Q147. [4 marks]

Evaluate the benefits and potential drawbacks of adopting Apache Iceberg over traditional Hive-managed tables for a large, multi-team data lake.

Ans:

- Among the benefits are safe concurrent schema evolution, ACID guarantees that prevent partial or corrupted reads during concurrent writes, and efficient query planning enabled by metadata-based file pruning. (2)
- Additional benefits include time travel and rollback capabilities, which support auditing and recovery from erroneous writes, along with hidden partitioning, which simplifies query writing for analysts. (1)
- Among the drawbacks are the added overhead of managing Iceberg's metadata, such as manifest files and snapshots, and the additional operational complexity of tasks like periodic snapshot expiration and compaction. (1)
- A further drawback is that adopting Iceberg requires supporting engines and tooling, and migrating existing Hive tables to Iceberg involves real, non-trivial migration effort for the team.

Q148. [4 marks]

Analyze the performance implications of Druid's roll-up during ingestion for a use case requiring exact per-event (non-aggregated) queries later.

Ans:

- Roll-up aggregates rows sharing identical dimensions and timestamp granularity, which means individual event-level detail is permanently discarded once aggregation occurs. (1)
- This significantly reduces storage requirements and improves aggregate query speed, but it also makes it impossible to later retrieve the exact original individual events. (1)
- If exact event-level queries are genuinely required, roll-up must be disabled, or the time granularity must be set extremely fine, which trades away much of the storage and performance benefit in order to retain that detail. (1)
- A hybrid approach — storing raw event logs separately, for example in a data lake, while using Druid with roll-up enabled for fast aggregate analytics — can balance both requirements effectively. (1)

Q149. [4 marks]

Compare Copy-on-Write and Merge-on-Read Hudi tables in terms of write amplification and query engine compatibility.

Ans:

- Copy-on-Write tables experience higher write amplification, since every update rewrites the entire affected base file, such as a whole Parquet file, even for a very small change. (1)
- Merge-on-Read tables have lower write amplification, since updates are appended as small delta log files rather than triggering a rewrite of the full base files. (1)
- Copy-on-Write tables are simpler to query, since any Parquet-compatible engine can read them directly without needing any special merge logic. (1)
- Merge-on-Read tables instead require query engines with Hudi-aware read support, capable of performing snapshot queries that correctly merge base and log files at read time, which limits their compatibility with more generic engines. (1)

Q150. [4 marks]

Critically assess the risks of tightly coupling business logic inside Airflow DAG files versus keeping DAGs as thin orchestration layers.

Ans:

- Embedding heavy business logic directly inside a DAG file means that logic re-executes on every Scheduler parse cycle if it sits at the top level, which degrades overall scheduler performance. (1)
- This tight coupling also makes the business logic harder to unit test independently of the Airflow framework, increasing development and debugging complexity. (1)
- Keeping DAGs as thin orchestration layers, which simply call out to external scripts, containers, or services, improves testability, reusability, and scheduler performance. (1)
- However, this thinner approach adds an extra layer of indirection and requires careful interface design, for example via XComs or external state, to correctly pass data between the orchestration layer and the actual business logic. (1)